

JAVASCRIPT - ARRAY-METHODEN

Frontend Tag 17

reduce() verstehen und sicher anwenden

Vom Array zu einer Summe, einem Objekt oder einem neuen Array - mit dem richtigen Startwert und einem klaren Akkumulator.

Das Lernziel

Du kannst erklären, was der Akkumulator macht, passende Startwerte wählen und **filter()** -> **map()** -> **reduce()** sinnvoll kombinieren.

01 Zahlen

Werte schrittweise addieren

02 Objekte

Properties summieren oder aufbauen

03 Arrays

Elemente gezielt sammeln

04 Chaining

auswählen, verändern, zusammenfassen

1 - reduce() mit Zahlen

reduce() verarbeitet jedes Element eines Arrays und trägt ein Zwischenergebnis weiter. Dieses Zwischenergebnis heißt Akkumulator.

```
const prices = [5, 10, 15, 20];

const totalPrice = prices.reduce((sum, number) => {
  return sum + number;
}, 0);

console.log(totalPrice); // 50
```

Startwert und Akkumulator

Durchlauf	Akkumulator sum	Aktueller Wert	Neues Ergebnis
1	0	5	5
2	5	10	15
3	15	15	30
4	30	20	50

Merksatz

Der Callback erhält zuerst den **Akkumulator** und danach das **aktuelle Element**. Der zurückgegebene Wert wird zum Akkumulator des nächsten Durchlaufs.

2 - Arrays aus Objekten

Ist das aktuelle Element ein Objekt, greifst du über eine Property auf den benötigten Wert zu - zum Beispiel **product.price** oder **course.hours**.

```
const products = [
  { name: "Tastatur", price: 50 },
  { name: "Maus", price: 25 },
  { name: "Monitor", price: 200 }
];

const totalPrice = products.reduce((sum, product) => {
  return sum + product.price;
}, 0);

console.log(totalPrice); // 275
```

Eine andere Property summieren

```
const courses = [
  { title: "HTML", hours: 5 },
  { title: "CSS", hours: 8 },
  { title: "JavaScript", hours: 12 }
];

const totalHours = courses.reduce((sum, course) => {
  return sum + course.hours;
}, 0);

console.log(totalHours); // 25
```

Typischer Fehler

products.price wäre falsch: **products** ist das ganze Array. Der Callback-Parameter **product** steht für das aktuelle einzelne Objekt.

3 - Mit reduce() ein Objekt aufbauen

Soll das Ergebnis ein Objekt sein, startest du mit `{}`. In jedem Durchlauf ergänzt du eine Property und gibst das Objekt wieder zurück.

```
const tools = ["Git", "GitHub", "VS Code"];

const toolStatus = tools.reduce((result, tool) => {
  result[tool] = "gelernt";
  return result;
}, {});

console.log(toolStatus);
```

```
{
  Git: "gelernt",
  GitHub: "gelernt",
  "VS Code": "gelernt"
}
```

Warum return result?

Ohne **return result** erhält der nächste Durchlauf nicht das aufgebaute Objekt. Bei einer Block-Funktion mit geschweiften Klammern musst du den Akkumulator ausdrücklich zurückgeben.

Startwert bestimmt Ergebnisform

`0` führt zu einer Zahl. `{}` führt zu einem Objekt. `[]` führt zu einem Array.

4 - Mit reduce() ein Array aufbauen

Mit `[]` als Startwert kannst du passende Elemente sammeln. Das kann ähnliche Aufgaben wie `filter()` lösen und ist besonders nützlich, wenn Auswahl und Umformung gemeinsam passieren.

```
const mySkills = ["HTML", "CSS", "JavaScript", "React", "Git"];

const longSkills = mySkills.reduce((result, skill) => {
  if (skill.length > 4) {
    result.push(skill);
  }

  return result;
}, []);

console.log(longSkills);
// ["JavaScript", "React"]
```

Die drei typischen Startwerte

Startwert	Ergebnis	Typischer Einsatz
0	Zahl	Summen, Zähler
{}	Objekt	Gruppieren, Lookup, Status
[]	Array	Sammeln, auswählen, umformen

Faustregel

Wähle den Startwert so, wie das fertige Ergebnis aussehen soll. Das macht den Code vorhersehbar und funktioniert auch bei einem leeren Eingabe-Array.

5 - filter() -> map() -> reduce()

Method Chaining liest sich wie eine kleine Daten-Pipeline: zuerst auswählen, dann verändern, schließlich zu einem Ergebnis zusammenfassen.

FILTER	MAP	REDUCE
auswählen	verändern	zusammenfassen
Welche Elemente bleiben?	Wie sieht jedes Element aus?	Was ist das Endergebnis?

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers
  .filter(number => number > 3)
  .map(number => number * 2)
  .reduce((sum, number) => sum + number, 0);

console.log(result); // 30
```

Ablauf

[1, 2, 3, 4, 5, 6] -> filter -> [4, 5, 6] -> map -> [8, 10, 12] -> reduce -> 30

6 - Abschlussübung: orders

Ziel: Nur Bestellungen über 20 Euro behalten, daraus die Preise holen und diese Preise addieren.

```
const orders = [
  { product: "Tastatur", price: 50 },
  { product: "Maus", price: 25 },
  { product: "Monitor", price: 200 },
  { product: "Kabel", price: 10 }
];

const total = orders
  .filter(order => order.price > 20)
  .map(order => order.price)
  .reduce((sum, price) => {
    return sum + price;
  }, 0);

console.log(total); // 275
```

Schritt für Schritt

1 - filter	Tastatur, Maus und Monitor bleiben. Kabel (10) fällt weg.
2 - map	Aus den Objekten werden nur die Preise: [50, 25, 200] .
3 - reduce	Der Startwert ist 0: 0 + 50 + 25 + 200 = 275 .

Fehlercheck

Im Callback immer auf das **aktuelle Element** zugreifen: **order.price**, nicht **orders.price**. Bei der Summe gehört nur der passende Startwert **0** an `reduce()`.

Kurzcheck für die Praxis

Bevor du `reduce()` ausführst, beantworte diese vier Fragen:

1**Was soll am Ende entstehen?**

Zahl, Objekt oder Array?

2**Welcher Startwert passt?**0, `{}` oder `[]`?**3****Was ist das aktuelle Element?**

Zahl, String oder Objekt?

4**Gebe ich den Akkumulator zurück?**Bei Block-Syntax: `return` nicht vergessen.

Tag 17 geschafft

Du hast `reduce()` mit Zahlen, Objekt-Arrays und verschiedenen Ergebnisformen eingesetzt - und die Methode mit `filter()` und `map()` zu einer verständlichen Pipeline kombiniert.